Web Application Penetration Test:

Broken Access Control (BAC) Impact Report

Vulnerability Details (Applicable to All Labs):

Vulnerability Name / Type:
Broken Access Control

Description:
Broken Access Control occurs when a web application fails to properly enforce restrictions on what authenticated or unauthenticated users can do. This allows users to access resources, data, or functionality they are not authorized to view or modify. In these labs, the application incorrectly exposes pages, functions, or data that should be restricted, enabling users to bypass intended security policies.

Impact / Risk:
Exploiting Broken Access Control can lead to unauthorized data access, privilege escalation, modification of sensitive information, or administrative actions by an attacker. This vulnerability significantly increases the risk of account compromise and system manipulation.

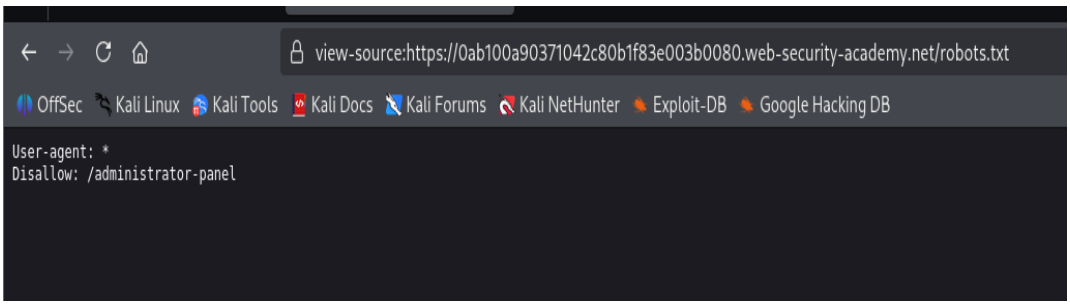
Lab 1:

Lab Title:
Unprotected admin functionality

Step 1:

Explanation:By using robots.txt file , and this User-agent appeared

Screenshot:

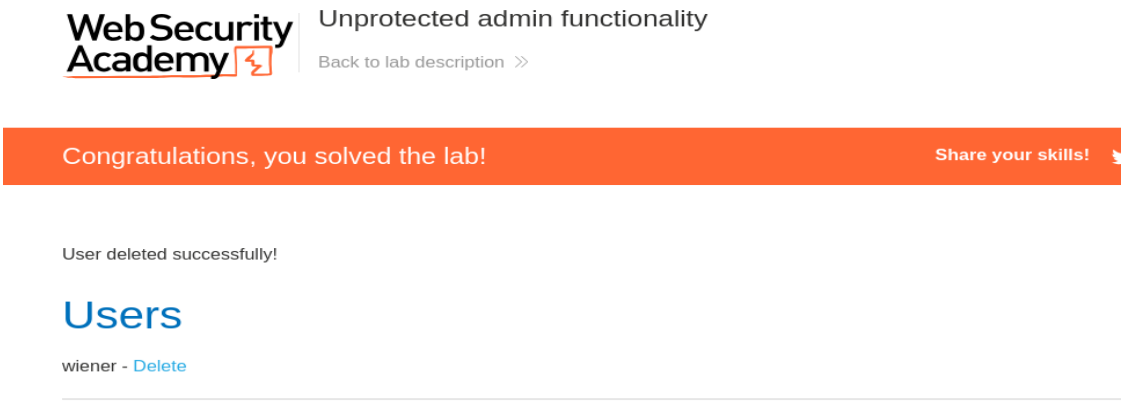


Step 2:

Explanation:

After wrote the whole path, the admin panel opened, then I deleted Carlos successfully

Screenshot:



Lab 2:

Lab Title:

Unprotected admin functionality with unpredictable URL

Step 1:

Explanation:

By taking a request and send it in the repeater, I found a script tag that has an end point that will lead me to the admin panel

Screenshot:

```
46              <div class="container is-page">
47                  <header class="navigation-header">
48                      <section class="top-links">
49                          <a href=/>Home
                        </a>
                        <p>
                            |
                        </p>
50                      <script>
51                          var isAdmin = false;
52                          if (isAdmin) {
53                              var topLinksTag = document.getElementsByClassName("top-links")[0];
54                              var adminPanelTag = document.createElement('a');
55                              adminPanelTag.setAttribute('href', '/admin-bt9jme');
56                              adminPanelTag.innerText = 'Admin panel';
57                              topLinksTag.append(adminPanelTag);
58                              var pTag = document.createElement('p');
59                              pTag.innerText = '|';
60                              topLinksTag.appendChild(pTag);
61                          }
62                      </script>
63                      <a href="/my-account">
                            My account
                        </a>
```

Step 2:

Explanation:

After wrote it in the URL the admin panel opened and deleted Carlos to solve the lab

Screenshot:

Lab 3:

Lab Title:

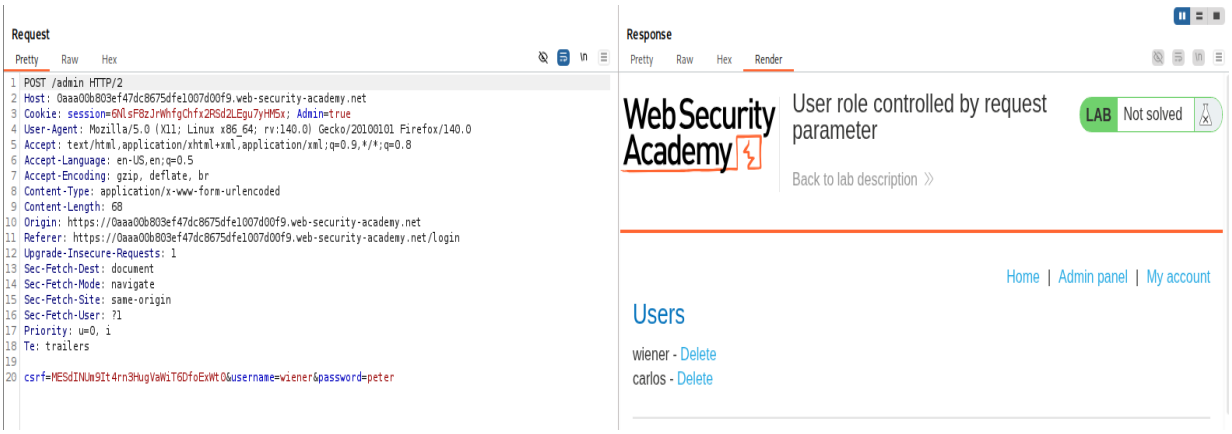
User role controlled by request parameter

Step 1:

Explanation:

By taking the login post request, I change the endpoint into: /admin, and there was a parameter called ADMIN, by changing it into true the admin panel has opened

Screenshot:

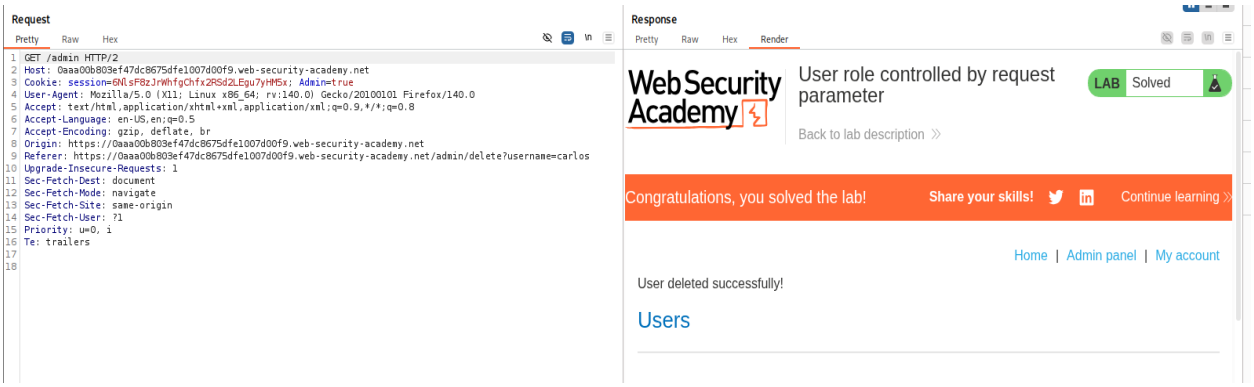


Step 2:

Explanation:

By adding the rest of the URL: /delete?username=carlos , Carlos deleted successfully

Screenshot:



Lab 4:

Lab Title:

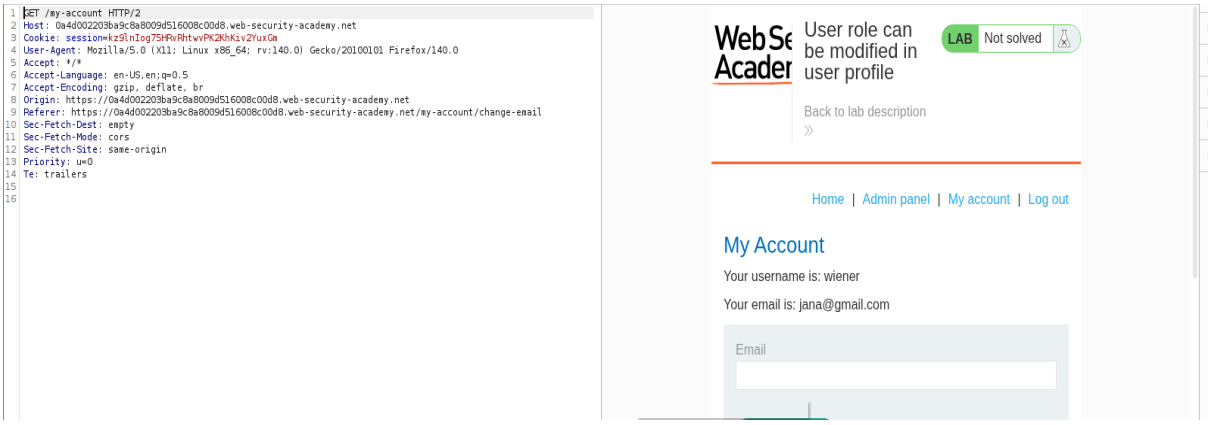
User role can be modified in user profile

Step 1:

Explanation:

As the lab description told me that the admin panel is only accessible to logged-in users with a roleid of 2, so after adding "roleid":2 into the JSON in the request body and resend it. It opened the admin panel page

Screenshot:

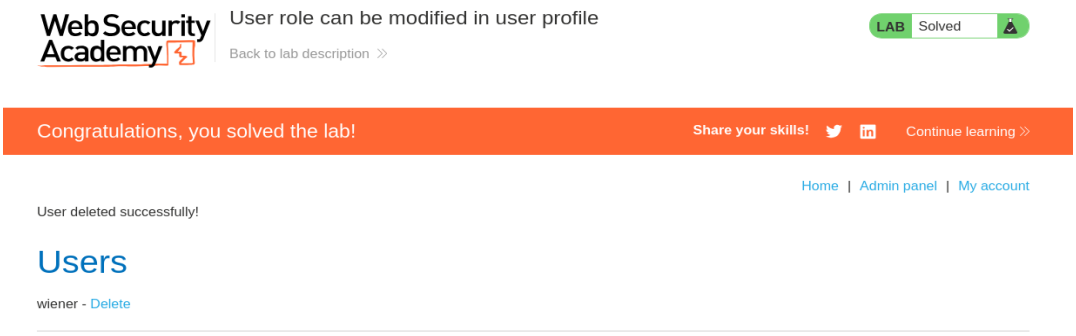


Step 2:

Explanation:

By opened it in the browser and deleted ,carlos the lab solved

Screenshot:



Lab 5:

Lab Title:

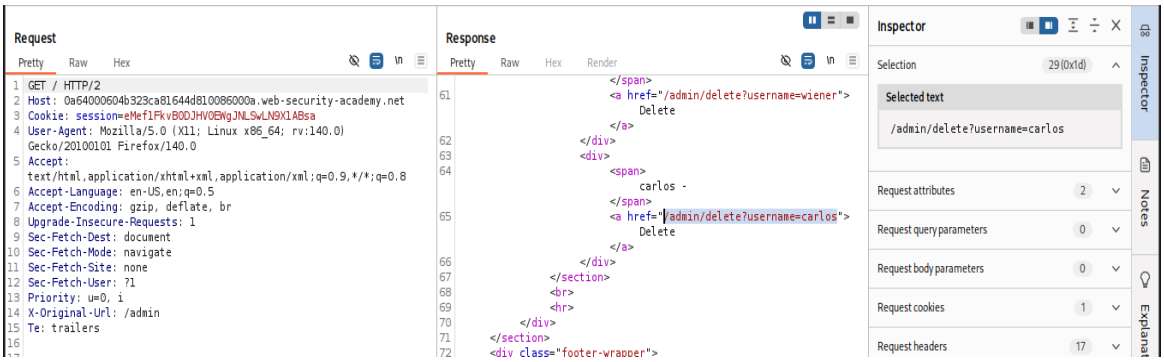
URL-based access control can be circumvented

Step 1:

Explanation:

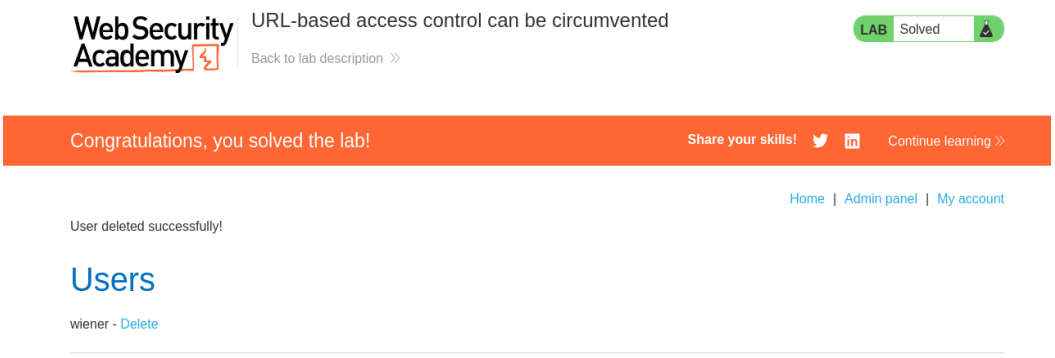
By adding this header to the body of the request: X-Original-Url: /admin, and making the URL request line empty like this /, it successfully opened the admin panel page

Screenshot:



Step 2:

Explanation:

By adding the rest of the URL which saying deleting Carlos, the lab successfully solved

Screenshot: